



PROJE GEREKSİNİMLERİ DOKÜMANI (PRD)

SentinelWatch Lite — Basitleştirilmiş Sistem İzleme ve Anomali Tespit Paneli

Süre: 3 Hafta — Tarih: Temmuz 2026

Birim: Veri Analitiği

1. Proje Amacı ve Vizyonu

SentinelWatch Lite, bir sisteme ait olay kayıtlarını (log/event) periyodik olarak izleyen, basit ama gerçek kurallara göre şüpheli davranışları (ör. brute-force giriş denemesi, anormal istek yoğunluğu) tespit eden ve bunları düzenli aralıklarla güncellenen bir dashboard üzerinde gösteren bir izleme panelidir.

Bu proje kasıtlı olarak sade tutulmuştur: az sayıda hareketli parça, net bir kapsam ve 3 haftaya rahatça sığacak, uçtan uca tamamlanabilir bir sistem. Tasarımda üç temel karar var: (1) WebSocket yerine periyodik sorgu (polling) kullanılıyor, (2) veritabanı tek bir tabloya indiriliyor, (3) 'kod değiştirmeden kural ekleme' fikri yerine backend içinde sabit yazılmış birkaç basit kontrol fonksiyonu kullanılıyor. Bu kararların her birinin gerekçesi Bölüm 3'te açıklanmıştır.

Proje süresince Sözleşme Öncelikli Geliştirme (Contract-First Development) ilkesi korunacak; backend ve frontend, üzerinde anlaşıkları iki basit REST endpoint sayesinde birbirini beklemeden paralel çalışabilecektir.

2. Teknoloji Yığını ve Rol Dağılımı

2.1 Karahan — Backend ve Anomali Tespiti

Sorumluluk Alanı: Olayların üretilmesi, tek bir tabloda saklanması ve basit kontrol fonksiyonlarıyla anlık olarak değerlendirilmesi.

- FastAPI (Python) ya da ASP.NET Core (.NET) ile basit bir Web API — hangisi Karahan için daha rahatsa o seçilebilir, iki seçenek de bu ölçekte eşdeğer zorlukta.
- Tek bir Events tablosu (SQLite yeterli — ayrı bir veritabanı sunucusu kurmaya gerek yok).
- Arka planda çalışan basit bir döngü (thread/timer) ile her 2 saniyede bir rastgele ama gerçekçi bir sahte olay (Mock Event) üretilip tabloya yazan bir fonksiyon.
- 2-3 adet sabit, doğrudan kod içinde yazılmış anomali kontrol fonksiyonu (bkz. Bölüm 4) — ayrı bir 'kural motoru' veya 'kural tablosu' yok.
- İki basit REST endpoint: /api/events ve /api/alerts (bkz. Bölüm 5).
- Geçersiz isteklerde uygun HTTP hata kodu dönülmesi.

2.2 Yağız — Frontend ve Görselleştirme

Sorumluluk Alanı: Backend'den periyodik olarak veri çekip ekranı güncel tutmak, olayları ve alarmları anlaşılır biçimde göstermek.

- React (Vite) + Tailwind CSS ile dashboard arayüzü — TypeScript zorunlu değil, JavaScript ile de rahatlıkla yapılabilir.

- setInterval ile her 5 saniyede bir /api/events ve /api/alerts endpoint'lerinin çağrılması (polling) — WebSocket, socket kütüphanesi ya da bağlantı yönetimi yok.
- Olay listesi paneli: en son gelen olaylar üstte görünecek şekilde basit bir liste.
- Recharts ile tek bir grafik: saatlik/dakikalık olay sayısı çizgi grafiği.
- Aktif bir alarm varsa ekranın üstünde kırmızı/turuncu bir uyarı kutusu gösterilmesi.
- Basit bir filtre: olay tipine göre listeyi daraltma (dropdown ile).

3. Sistem Mimarisi ve Tasarım Kararları

Aşağıdaki akış, az sayıda bileşenden oluşan uçtan uca bir döngüyü gösterir:

1. Backend içinde çalışan basit bir döngü, her birkaç saniyede bir sahte bir olay üretir ve Events tablosuna yazar.
2. Frontend, her 5 saniyede bir /api/events endpoint'ini çağırarak son olayları çeker ve listeyi/grafiği günceller.
3. Frontend, aynı zamanda /api/alerts endpoint'ini de çağırır; backend bu istekte, Events tablosunu o an sorgulayıp 2-3 basit kontrolü çalıştırır ve varsa aktif tehlike durumlarını döner.
4. Yani alarm, ayrı bir tabloda saklanan bir kayıt değil; her istek geldiğinde 'şu an aktif mi?' diye anlık hesaplanan bir sonuçtur.

Tasarım Kararları ve Gerekçeleri

Konu	Değerlendirilen Alternatif	Seçilen Yaklaşım
Veri iletimi	WebSocket / SignalR (anlık push)	5 saniyede bir polling (fetch)
Veritabanı	3 tablo (Events, Rules, Alerts)	1 tablo (Events) — alarm anlık hesaplanır
Kural yönetimi	DB'den okunan, dinamik kural motoru	Kod içinde 2-3 sabit fonksiyon
AI katmanı	Opsiyonel LLM günlük özeti	Yok (kapsam dışı)

Kullanıcı deneyimi açısından fark neredeyse hissedilmez — dashboard hâlâ kendiliğinden güncelleniyor, hâlâ alarm üretiliyor. Ama Yağız 'bağlantı koptu, yeniden bağlan' gibi durumlarla uğraşmıyor; Karahan üç tablo arasında foreign key ilişkisi kurmak yerine tek bir tabloyu iyi tasarlamaya odaklanıyor. 3 haftalık süre için doğru denge budur.

4. Veritabanı ve Anomali Kontrolleri

4.1 Events Tablosu (Tek Tablo)

Alan	Tip	Açıklama
id	integer (PK)	Otomatik artan kimlik
timestamp	datetime	Olayın gerçekleştiği zaman
source_ip	string	Olayın kaynağı olan IP adresi
event_type	string	LOGIN_FAILED, LOGIN_SUCCESS, HIGH_CPU, REQUEST vb.
username	string (opsiyonel)	İlgiliyse kullanıcı adı

Bu kadar. Ayrı bir Rules ya da Alerts tablosu yok — alarm mantığı tamamen backend kodunda yaşıyor.

4.2 Anomali Kontrol Fonksiyonları (Kod İçinde, Sabit)

Karahan, /api/alerts çağrıldığında sırayla çalışan 2-3 basit fonksiyon yazar. Her fonksiyon, son birkaç dakikalık veriyi Events tablosundan çeker ve basit bir koşulu kontrol eder.

Kontrol Adı	Mantık (basit Türkçe)	Alarm Seviyesi
check_brute_force()	Son 5 dakikada aynı IP'den 5'ten fazla LOGIN_FAILED var mı?	Yüksek
check_traffic_spike()	Son 1 dakikada toplam olay sayısı 100'ü geçti mi?	Orta
check_high_cpu()	Son 2 dakikada 3'ten fazla HIGH_CPU olayı var mı?	Düşük

```
# Python / FastAPI örneği – gerçek kod böyle sade kalabilir
def check_brute_force(db):
    five_min_ago = now() - timedelta(minutes=5)
    rows = db.query(Event).filter(
        Event.event_type == 'LOGIN_FAILED',
        Event.timestamp >= five_min_ago,
    ).all()
    counts = {}
    for r in rows:
        counts[r.source_ip] = counts.get(r.source_ip, 0) + 1
    alerts = []
    for ip, count in counts.items():
        if count >= 5:
            alerts.append({
                'type': 'BRUTE_FORCE',
                'severity': 'high',
                'sourceIp': ip,
                'description': f'{ip} adresinden 5 dakikada {count} basarisiz giris'
            })
    return alerts
```

Bu fonksiyonlar kasıtlı olarak 'ilkel' tutulmuştur — amaç Karahan'a karmaşık bir kural motoru mimarisi değil, temiz ve okunabilir bir sorgu + basit Python/C# mantığı yazdırmaktır. Zaman kalırsa üçüncü haftada bu fonksiyonlar küçük bir refactor ile ortak bir yapıya kavuşturulabilir; bu isteğe bağlıdır.

5. API Sözleşmesi

Sadece iki endpoint var. Bu sözleşme, 1. Hafta'nın ilk gününde birlikte netleştirilmeli ki Yağız mock veriyle beklemeden ilerleyebilsin.

5.1 GET /api/events

Son olayları, en yeniden en eskiye sıralı biçimde döner. Basitlik için sayfalama zorunlu değildir; son 100 kayıt yeterlidir.

```
// Örnek yanıt
[
  {
    "id": 4213,
    "timestamp": "2026-07-06T14:32:10Z",
    "sourceIp": "185.23.11.4",
    "eventType": "LOGIN_FAILED",
    "username": "admin"
  },
  ...
]
```

```
{
  "id": 4212,
  "timestamp": "2026-07-06T14:32:08Z",
  "sourceIp": "91.44.10.2",
  "eventType": "REQUEST",
  "username": null
}
```

5.2 GET /api/alerts

Bu istek geldiğinde backend, Bölüm 4.2'deki kontrol fonksiyonlarını sırayla çalıştırır ve o an aktif olan tehlike durumlarını döner. Hiçbir şey tetiklenmiyorsa boş liste döner.

```
// Örnek yanıt
[
  {
    "type": "BRUTE_FORCE",
    "severity": "high",
    "sourceIp": "185.23.11.4",
    "description": "185.23.11.4 adresinden 5 dakikada 6 basarisiz giris denemesi"
  }
]
```

5.3 Frontend Tarafı — Polling Örneği

```
// React – basit polling (WebSocket yok)
useEffect(() => {
  const fetchData = async () => {
    const events = await fetch('/api/events').then(r => r.json());
    const alerts = await fetch('/api/alerts').then(r => r.json());
    setEvents(events);
    setAlerts(alerts);
  };
  fetchData();
  const interval = setInterval(fetchData, 5000);
  return () => clearInterval(interval);
}, []);
```

6. 3 Haftalık Proje Yol Haritası

Hafta 1 — Sözleşme ve İskelet (ORTAK BAŞLANGIÇ)

- Gün 1 (Birlikte): /api/events ve /api/alerts için JSON formatı netleştirilir (Bölüm 5).
- Karahan: API iskeleti, Events tablosu, Mock Event Generator (her 2 saniyede bir sahte olay üretimi).
- Yağız: React proje iskeleti, mock JSON veriyle statik ekran tasarımı (liste + grafik alanı + alarm kutusu yerleşimi).
- Hafta sonu hedefi: Backend gerçek veri üretip /api/events üzerinden döndürüyor; frontend mock veriyle tüm ekran iskeleti hazır.

Hafta 2 — Gerçek Bağlantı ve İlk Kontroller (PARALEL GELİŞTİRME)

- Karahan: check_brute_force() ve check_traffic_spike() fonksiyonlarının yazılması, /api/alerts endpoint'inin gerçek veriyle çalışır hale gelmesi.
- Yağız: Mock veri yerine gerçek /api/events ve /api/alerts çağrılarına geçiş (polling), olay listesinin canlı güncellenmesi, ilk grafik entegrasyonu.

- Hafta sonu hedefi: Sistem uçtan uca çalışıyor — backend'de üretilen bir olay, birkaç saniye içinde ekranda görünüyor; en az bir kontrol gerçek alarm üretebiliyor.

Hafta 3 — Cilalama ve Kapanış (ENTEGRASYON DÖNEMİ)

- Karahan: `check_high_cpu()` eklenir, hata yönetimi (geçersiz istek, boş veri durumları) tamamlanır.
- Yağız: Alarm kutusunun renk kodlaması, olay tipine göre filtre, responsive düzenlemeler.
- Karahan ve Yağız (Birlikte): Uçtan uca test (sistemi çalıştır, brute-force senaryosunu gözlemle, alarmin doğru görüldüğünü doğrula), README.md yazımı, son kod incelemesi ve PR birleştirme.
- Hafta sonu hedefi: Sistem demo edilebilir durumda; en az 2 kontrol aktif, dashboard canlı çalışıyor, README ile sıfırdan kurulum mümkün.

7. Başarı Kriterleri (Definition of Done)

- Git & Branch Yönetimi: main branch'ine doğrudan kod basılamaz; her stajyer kendi branch'inde çalışır, PR ile birleştirme yapılır.
- Uçtan Uca Çalışan Akış: Mock Event Generator'dan üretilen bir olay veritabanına yazılmalı ve en geç 5 saniye içinde frontend'de görünmelidir.
- En Az 2 Aktif Kontrol: `check_brute_force()` ve en az bir kontrol daha çalışır ve doğru koşullarda alarm üretir durumda olmalıdır.
- Gelişmiş Hata Yönetimi: Geçersiz istekte backend çökmemeli, uygun HTTP durum kodu dönmelidir; frontend hataları kullanıcıya sade biçimde göstermelidir.
- Temiz Dokümantasyon: Projenin kök dizininde, projenin sıfırdan nasıl ayağa kaldırılacağını (bağımlılıklar, veritabanı dosyası, Mock Generator'ın başlatılması) anlatan bir README.md bulunmalıdır.
- Demo Senaryosu: Ekip, sistemi çalıştırıp bir brute-force simülasyonunun birkaç saniye içinde ekranda alarma dönüştüğünü canlı gösterebilmelidir.

8. Riskler ve Dikkat Edilmesi Gereken Noktalar

- Polling Aralığı: 5 saniye çoğu senaryo için yeterlidir; çok kısa tutulursa (ör. 1 saniye) backend'e gereksiz yük biner, bu konuda abartıya gerek yoktur.
- Tek Tablo Yeterliliği: 'Alarm geçmişini de saklamak istersek?' sorusu gelirse, bu bilinçli bir kapsam dışı bırakma kararıdır; istenirse gelecekte küçük bir Alerts tablosu eklenebilir, ama 3 haftalık kapsamda gerekli değildir.
- Kapsam Genişlemesi: 'Bir de şunu ekleyelim' isteklerine karşı Definition of Done listesi referans alınmalı; ekstra fikirler not edilip 3. haftadan sonraya bırakılmalıdır.